

TD d'Algorithmique et structures de données

Arbres Binaires & Arbres Binaires de Recherche

Équipe pédagogique Algo SD

Soit A un arbre binaire. Il est représenté par les structures de données suivantes (en python) :

```
class Arbre:
    def __init__(self):
        self.racine = None

class Noeud:
    def __init__(self, valeur, fils_gauche=None, fils_droit=None):
        self.valeur = valeur
        self.fils = [fils_gauche, fils_droit]
```

1 Image Miroir d'un arbre binaire

1. Étant donné un arbre binaire A , écrire un algorithme qui calcule le reflet de A (les fils gauches deviennent les fils droits, et ceci dans chaque sous arbre). Écrire deux versions, l'une – fonctionnelle – renvoyant une copie, l'autre – procédurale – modifiant en place la structure.
2. Donner le coût en nombre d'affectations de l'algorithme, en fonction du nombre d'éléments n .

2 Parcours arborescents

On considère un jeu de stratégie à deux joueurs en tour par tour. On dispose d'une classe *Plateau* qui stocke l'état courant du plateau de jeu (la position des pièces et l'identifiant du joueur courant). Sur cette classe deux méthodes sont disponibles :

- *coups(self)* qui renvoie un itérateur sur tous les coups que le joueur courant peut jouer ;
- *joue_coup(self, coup)* qui crée un nouveau *Plateau* applique le coup donné et renvoie la nouvelle configuration (pour l'autre joueur donc) sans modifier *self*.

1. Proposez une fonction récursive qui renvoie le nombre de plateaux atteignables en jouant une séquence de s coups.
2. Proposez une fonction itérative qui renvoie le nombre de plateaux atteignables en jouant une séquence de s coups.
3. Proposez une fonction itérative renvoyant un itérateur sur tous les plateaux visités lors d'un parcours en profondeur, d'une profondeur de s .
4. Proposez une fonction simple (une ligne) permettant à l'aide de l'itérateur de la question précédente de compter le nombre de plateaux différents accessibles depuis le plateau de départ avec un parcours de profondeur s .
5. On se propose d'optimiser ce programme à l'aide de coupes (en évitant les sous-arbres identiques). Proposez deux versions, une avec un parcours en profondeur et une autre avec un parcours en largeur. Comment la détection des doublons change-t-elle suivant le type de parcours ?

3 Vérification qu'un arbre binaire est un ABR

1. Ecrire en Python ou en pseudo-code la fonction récursive *VERIF(self)* qui vérifie si un arbre binaire est un ABR, et dans ce cas renvoie les éléments minimum et maximum.

4 Suppression dans un ABR

On s'intéresse à la suppression d'un élément dans un ABR. Comme pour l'insertion, elle nécessite de mettre à jour la structure de l'arbre. Les modifications doivent être réalisées de manière à conserver les invariants de structure et d'ordre.

1. Commencez par essayer quelques exemples de suppression manuellement. Y-a-t'il des cas "faciles" ?
2. Écrire en Python ou en pseudo-code un algorithme `DELETE(self, key)` qui supprime un élément *key* dans un ABR.
3. Donner le coût (en meilleur cas, en pire cas) de la primitive DELETE en fonction du nombre d'éléments *n* dans l'arbre.

5 Sous-ensemble d'un ABR

1. Écrire en python ou en pseudo-code un algorithme `ITERATE(Min, Max, A)` qui itère sur les éléments de *A* entre *Min* et *Max*.